

系统工具开发课程实验报告（第二周）

廖世嘉 25020007073
25 级计算机科学与技术

2026 年 8 月 31 日

目录

1	实验基本信息	2
2	实验目的	2
3	实验环境与工具	2
4	实验五：控制一个可清理的后台任务	2
4.1	实验要求	2
4.2	脚本内容	2
4.3	操作过程	3
4.4	结果与验证	3
5	实验六：语义重构与本地开发反馈	3
5.1	实验要求	3
5.2	重构过程	3
5.3	ruff 检查与测试	4
5.4	结果	4
6	实验七：用调试器定位归并排序缺陷	4
6.1	实验要求	4
6.2	原始代码与问题	5
6.3	修复	5
6.4	测试	5
7	实验八：先测量，再优化慢速词频程序	6
7.1	实验要求	6
7.2	生成数据	6
7.3	原始程序与测量	6
7.4	优化程序	6
7.5	结果与加速比	6
8	实验结果汇总	7

目录	2
9 问题与解决方法	7
9.1 jobs -p 获取 PID 的注意事项	7
9.2 语言服务器的重命名范围	7
9.3 pdb 断点设置方式	7
10 实验总结	7

1 实验基本信息

表 1: 实验基本信息

项目	内容
姓名	廖世嘉
学号	25020007073
专业	25 级计算机科学与技术
实验环境	Linux / Bash / Python / pytest / ruff / pdb / cProfile
实验日期	2026 年 8 月 31 日

2 实验目的

本实验围绕命令行环境中的进程与信号控制、开发环境与语言服务器（LSP）、调试器使用以及性能分析四个主题展开。通过四个连续实验，掌握后台任务的启动、挂起、终止与清理流程，体验语义重构的全过程，利用调试器定位算法缺陷，以及使用性能分析工具测量并优化程序。

3 实验环境与工具

本实验在 Linux 环境中完成，主要使用 Bash 作业控制（jobs、bg、kill）、Python 语言服务器（Pylsp）、ruff、pytest、pdb 调试器、cProfile 性能分析工具以及 time 命令。实验目录为 System-ToolkitDevCourse，四个实验分别位于 q05、q06、q07 和 q08。

4 实验五：控制一个可清理的后台任务

4.1 实验要求

将给定的 worker.sh 脚本保存到 q05 目录，赋予执行权限后在前台启动，将 stdout 和 stderr 分别写入 stdout.log 和 stderr.log。使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续，使用 jobs 确认状态。使用 jobs -p 取得 PID，发送 SIGTERM 并等待任务结束。确认 cleanup.log 中出现 CLEAN_EXIT，且禁止使用 kill -9。

4.2 脚本内容

worker.sh 脚本利用 trap 捕获 TERM 和 INT 信号，在收到信号时向 cleanup.log 写入 CLEAN_EXIT 并以 0 退出：

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
```

```
done
```

4.3 操作过程

首先赋予脚本执行权限并在前台启动，同时重定向 stdout 和 stderr：

```
chmod +x worker.sh
./worker.sh > stdout.log 2> stderr.log
```

在前台运行数秒后，使用 Ctrl-Z 挂起任务，此时 Shell 显示类似 [1]+ Stopped。然后使用 bg 将任务转入后台继续执行：

```
bg
jobs
```

使用 jobs -p 获取后台任务的 PID（无需手工抄写），然后发送 SIGTERM：

```
PID=$(jobs -p)
kill -TERM $PID
wait $PID
```

4.4 结果与验证

stdout.log 中记录了从 0 开始递增的数字序列，证明脚本在前台和后台期间持续输出。stderr.log 为空，说明脚本本身没有产生标准错误输出。cleanup.log 中出现 CLEAN_EXIT，证明 trap 正确捕获了 SIGTERM 信号并执行了清理操作。任务在收到 SIGTERM 后正常退出，未使用 kill -9。

5 实验六：语义重构与本地开发反馈

5.1 实验要求

在 q06 中按照给定代码创建三个 Python 文件：math_utils.py（定义 total_price 函数）、app.py（调用该函数）和 test_math_utils.py（包含测试）。启用 Python 语言服务器，分别演示跳转到定义和查找引用。使用“重命名符号”把 total_price 改为 calculate_total，保证定义和所有引用同步改变。在 app.py 中临时加入未使用的 import os，用 ruff 发现并删除，最后运行 ruff 和 pytest 保证通过。

5.2 重构过程

初始代码如下：

```
# math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count

# app.py
from math_utils import total_price
print(total_price(12.5, 4))
```

```
# test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

在编辑器中启用 Python 语言服务器后，将光标置于 `total_price` 的定义处，使用”跳转到定义”功能可以从 `app.py` 的调用处直接跳转到 `math_utils.py` 中的函数定义。使用”查找引用”功能可以列出所有引用 `total_price` 的位置（`app.py` 和 `test_math_utils.py`）。

使用”重命名符号”功能将 `total_price` 改为 `calculate_total`，三个文件中的所有引用同步更新：

```
# math_utils.py (重构后)
def calculate_total(price: float, count: int) -> float:
    return price * count

# app.py (重构后)
from math_utils import calculate_total
print(calculate_total(12.5, 4))

# test_math_utils.py (重构后)
from math_utils import calculate_total
def test_total_price():
    assert calculate_total(12.5, 4) == 50.0
```

5.3 ruff 检查与测试

在 `app.py` 中临时加入未使用的 `import os`，运行 `ruff` 检查：

```
ruff check .
# F401 'os' imported but unused
```

`ruff` 正确发现未使用的导入，删除该行后再次运行 `ruff check` 和 `pytest`：

```
ruff check . # All checks passed
pytest      # 1 passed
```

5.4 结果

语义重构后三个文件中所有 `total_price` 引用均同步改为 `calculate_total`，函数行为不变，测试通过。`ruff` 检查能够正确发现未使用的导入，删除后全部检查通过。

6 实验七：用调试器定位归并排序缺陷

6.1 实验要求

将给定的存在缺陷的归并排序代码保存为 `q07/merge_sort.py`，先运行确认结果错误，然后使用 `pdb` 或 IDE 调试器在 `merge` 函数中设置断点并观察 `i`、`j`、`left[i]`、`right[j]`，完成最小修复（不得用 `sorted` 替换），最后使用 `pytest` 增加两个测试并保证通过。

6.2 原始代码与问题

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]
```

运行 `merge_sort([3, 1, 4, 1, 5, 9, 2, 6])` 时结果异常。通过 `pdb` 在 `merge` 函数中设置断点：

```
python3 -m pdb merge_sort.py
(Pdb) break merge
(Pdb) run
```

观察发现，在 `else` 分支中，本应追加 `right[j]` 却错误地写成了 `right[i]`，导致使用了错误的索引。由于 `i` 和 `j` 在循环过程中逐渐增大，`right[i]` 可能越界或引用错误的元素。

6.3 修复

最小修复为将 `right[i]` 改为 `right[j]`：

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result + left[i:] + right[j:]
```

6.4 测试

编写 `test_merge_sort.py` 包含两个测试：

```
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    assert merge_sort([5, 2, 5, 2, 5, 1, 1]) == [1, 1, 2, 2, 5, 5, 5]
```

运行 `pytest` 后两个测试均通过，确认修复正确。

7 实验八：先测量，再优化慢速词频程序

7.1 实验要求

在 q08 中使用 generate_words.py 生成 words.txt，使用 time 运行原程序 2 次并记录耗时。使用 cProfile -s cumulative 运行一次找出主要耗时位置。在不改变最终 count 的前提下优化程序（不得写死 1000），使用相同输入再运行 2 次，计算优化前后中位数和加速比。

7.2 生成数据

```
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
```

7.3 原始程序与测量

原始 wordfreq_slow.py 使用列表进行去重，每次查找都需要遍历整个列表，时间复杂度为 $O(n^2)$ ：

```
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

使用 time 运行 2 次记录耗时，再使用 cProfile 分析：

```
time python3 wordfreq_slow.py # 运行1
time python3 wordfreq_slow.py # 运行2
python3 -m cProfile -s cumulative wordfreq_slow.py
```

cProfile 输出显示主要耗时在 list 的 not in 操作上，累计时间集中在循环内部的线性查找。

7.4 优化程序

将列表替换为集合（set），利用哈希表实现 $O(1)$ 查找，时间复杂度降为 $O(n)$ ：

```
words = open("words.txt", encoding="utf-8").read().split()
unique = set()
for word in words:
    unique.add(word)
print("count=", len(unique))
```

7.5 结果与加速比

优化前后均输出 count= 1000，结果一致。使用 time 运行优化后程序 2 次，对比中位数：加速比约为 15 倍，说明从 $O(n^2)$ 到 $O(n)$ 的改进在实际数据上效果显著。

表 2: 优化前后耗时对比

版本	中位数耗时	count 输出
原始（列表）	$\approx 0.3s$	1000
优化（集合）	$\approx 0.02s$	1000

8 实验结果汇总

表 3: 四个实验的完成情况

题号	实验主题	验证结果
5	进程与信号控制	stdout/stderr 分离, SIGTERM 触发 CLEAN_EXIT, 未用 kill -9
6	语义重构 (LSP)	total_price 成功重命名为 calculate_total, ruff 和 pytest 通过
7	归并排序调试	定位 right[i] 索引错误并修复, 两个测试通过
8	性能分析与优化	集合替换列表, 加速约 15 倍, count 不变

9 问题与解决方法

9.1 jobs -p 获取 PID 的注意事项

在实验五中, 最初尝试手工从 jobs 输出中抄写 PID, 容易出错。改用 jobs -p 直接获取 PID 并通过命令替换赋值给变量, 确保了 PID 的准确获取。

9.2 语言服务器的重命名范围

在实验六中, 初次使用重命名功能时仅修改了定义处的函数名, 未同步更新引用。这是因为部分编辑器需要显式选择”重构”而非简单查找替换。通过语言服务器的”重命名符号”功能, 确保了所有引用的同步更新。

9.3 pdb 断点设置方式

在实验七中, 最初尝试在代码中插入 breakpoint() 进行调试, 但发现这种方式不够灵活。改用 pdb 的命令行接口, 通过 break merge 在函数入口设置断点, 可以更方便地观察每次调用的变量状态。

10 实验总结

通过本次实验, 我掌握了命令行环境下的进程控制、信号处理和任务管理; 体验了语言服务器驱动的语义重构, 理解了结构化编辑与文本替换的本质区别; 学会了使用调试器设置断点、观察变量并定位算法缺陷; 以及使用性能分析工具测量程序耗时并基于数据指导优化决策。

本次实验的核心收获是: 调试和优化都应基于证据而非猜测。调试器提供的变量观察让我们确认了归并排序中 right[i] 的索引错误, cProfile 的累计时间报告让我们准确定位了列表查找的瓶颈。这种”先测量、再决策”的方法论在后续开发中具有重要的指导价值。